21CSE525T – AGENT TECHNOLOGY

ASSIGNMENT 4.1

Belief-Desire-Intention (BDI) Goal-Driven Modeling

A cognitive layer that re-plans CipherMesh intentions when beliefs change mid-execution

Submitted by

KUMAR WAIBHAV AKSHAT [RA2212704010027]

M.Tech (Integrated) Computer Science & Engineering
with specialization in Data Science

Under the Guidance of

Dr. Prabhu Kavin B

Assistant Professor
Department of Data Science and Business Systems



DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203

NOVEMBER 2026

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Objective 1
1.2 The problem in CipherMesh terms 1
1.3 How this unit builds on Units 1 to 3 1
2 ASSIGNMENT REQUIREMENTS AND TRACEABILITY 2
3 DESIGN 3
3.1 Architecture 3
3.2 Background: the BDI model 3
3.3 Beliefs 4
3.4 Desires 4
3.5 Plans and the plan library 5
3.6 Intentions and their life cycle 5
3.7 The deliberation cycle 6
3.8 Reconsideration and commitment strategy 7
3.9 Class and data model 8
4 IMPLEMENTATION 8
4.1 Modules 8
4.2 Selected code 9
4.3 Invariants enforced in code 10
5 VERIFICATION RUN 10
5.1 Set-up 10
5.2 Baseline: no disturbance (S0) 11
5.3 S1: provider outage mid-dispatch 11
5.4 S2: emergency lockdown mid-execution 12
5.5 S3: the provider reply leaks a secret 13
5.6 S4: price spike, then budget cut 13
5.7 S5 and S6: Indian data residency 13
5.8 S7 and S8: suspension and a shift across desires 14
5.9 Summary of all scenarios 15
5.10 Ablation: why reconsideration matters 16
6 TESTING 16
7 SUBMISSION ANALYSIS 17
7.1 Strengths 18
7.2 Limitations 18
7.3 How this feeds the Capstone (Assignment 5.2) 18
8 CONCLUSION 18
REFERENCES 19
App. A HOW TO RUN 20
App. B FILE LISTING 21

LIST OF FIGURES

Figure 3.1 BDI architecture of the CipherMesh cognitive layer (package ciphermesh/bdi).....3
Figure 3.2 Intention state machine: ACTIVE, SUSPENDED and the three terminal states.....6
Figure 3.3 Deliberation cycle with the re-plan path (top) and the next-cycle path (bottom).....6
Figure 5.1 S1 swimlane: intention I1 is dropped at the outage belief and I2 completes on AWS Bedrock..... 12
Figure 5.2 All runs on one cycle axis: steps per intention, disturbances, drops and suspensions.....15
Figure 5.3 Ablation counts from logs/bdi_summary.json.....16

LIST OF TABLES

Table 1.1 Earlier CipherMesh units reused by the BDI layer.....1
Table 2.1 Requirement traceability matrix.....2
Table 3.1 Belief families used by the CipherMesh agent.....4
Table 3.2 CipherMesh desires (desires.py).....5
Table 3.3 Plan library (plans.py).....5
Table 3.4 Commitment strategies implemented in engine.py.....7
Table 3.5 Class and data model of ciphermesh/bdi.....8
Table 4.1 Modules of the BDI subpackage and demonstration.....8
Table 5.1 Scenario catalogue and injection points (scenarios.py).....11
Table 5.2 Summary of the verification run (open-minded commitment).....15
Table 5.3 Ablation: open-minded versus blind commitment.....16
Table 6.1 BDI test files and what they cover.....17
Table 7.1 Requirement, evidence and status.....17
Table B.1 Files in the submission.....21

1 INTRODUCTION

1.1 Objective

This assignment adds a **cognitive layer** to CipherMesh, the privacy control plane that sits between enterprise applications and AI providers. The layer follows the Belief-Desire-Intention (BDI) model of practical reasoning [1] [2]. It keeps three explicit collections inside the agent: **beliefs** (what the agent currently holds to be true), **desires** (the strategic goals it would like to reach) and **intentions** (the step-by-step plans it has committed to). The objective set by the brief is to track these three collections as a state machine and to *recalculate the plan* when a belief changes while a plan is half-way through execution.

Earlier units made CipherMesh correct for one moment in time: a prompt was inspected, a provider was chosen and the prompt was sent. Real deployments do not stand still. A provider can go down between the routing decision and the transmission, the security team can declare a lockdown, a price can jump, or the user can edit the prompt. A purely reactive pipeline either sends the prompt anyway or fails late. A BDI agent instead notices that one of its working assumptions is no longer true, drops or suspends the affected intention, and chooses a new plan. This report shows that behaviour with real engine calls and real logs.

1.2 The problem in CipherMesh terms

For every prompt CipherMesh must balance goals that can conflict. It must never transmit a secret, it must keep Indian personal data (Aadhaar, PAN, Indian phone numbers) inside India or on the local host, it should still return a useful answer, it should do so at the lowest cost within quality limits, and it must not overload the human reviewers. These map directly to five desires D1 to D5 with fixed priorities. The question this unit answers is: *when the world changes in the middle of a request, which of these goals should drive the next action, and what plan should replace the one that is no longer valid?*

1.3 How this unit builds on Units 1 to 3

The BDI layer does not re-implement any engine logic. Every plan step calls a module built in an earlier unit, as listed in Table 1.1. This keeps the cognitive layer thin and makes the verification run a test of the whole product, not of a toy model.

Table 1.1: Earlier CipherMesh units reused by the BDI layer

Unit	Module(s)	Role inside the BDI layer
1.1 / 1.2	`detector.py`, `policy.py`	The prompt sensor: 25 detector rules and policy R1 to R5 produce the beliefs `prompt.decision` and `prompt.categories`
2.1	`runtime.py`, `providers.py`	Thread-safe engine and provider health; the belief base is also lock-protected so monitors can update it from other threads
2.2	`agent.py`, `security.py`, `audit.py`	Agent state (`force_block_all`), review queue, response inspection and the SQLite audit log that records each adopted and dropped intention
3.1	`procurement.py`, `negotiation/`	Contract-Net prices [3] become the belief `provider.<id>.price_per_1k` and are confirmed by the `procure_contract` step
3.2	`routing.py`	`route()` and `derive_constraints()` rank providers by U = 0.30Q + 0.25C + 0.20L + 0.15A + 0.10P; the ranking supplies plan bindings and their utilities

2 ASSIGNMENT REQUIREMENTS AND TRACEABILITY

Table 2.1 quotes every objective, direction and deliverable line of the brief and maps it to the place where it is met and to the evidence a reviewer can open. File paths are relative to the submission folder.

Table 2.1: Requirement traceability matrix

Brief line	Where it is satisfied	Evidence
Objective: Implement a cognitive-layer state machine tracking beliefs, desires, and execution intentions inside your custom agent.	`ciphermesh/bdi/engine.py` (BDIAgent deliberation cycle) and `intentions.py` (intention state machine)	Figures 3.2 and 3.3; `logs/bdi_trace.jsonl`
Construct three explicit collections or classes within your application architecture mapping to the BDI Architecture	`BeliefBase`, `DesireSet` and `PlanLibrary` + `IntentionStack` in `ciphermesh/bdi/`	Section 3.9, Table 3.5
Beliefs: The agent's knowledge base state repository (e.g., "Current Location battery is 20%", "Flight price is $400").	`ciphermesh/bdi/beliefs.py`: `Belief`, `BeliefBase` (set, get, has, retract, subscribe, snapshot, history, lock)	`tests/bdi/test_beliefs.py` (13 tests incl. a 12-thread stress test)
Desires: Top-level abstract strategic milestones (e.g., "Find high-speed network", "Book lowest cost ticket").	`ciphermesh/bdi/desires.py`: `Desire`, `DesireSet`, `ciphermesh_desires()` with D1 to D5	Table 3.2; `tests/bdi/test_desires.py`
Intentions: Active step-by-step procedural action pipelines chosen to hit active desires.	`ciphermesh/bdi/plans.py` (Plan, PlanStep, PlanLibrary) and `intentions.py` (Intention, IntentionStack)	Table 3.3; `tests/bdi/test_plans.py`, `test_intentions.py`
Program dynamic plan recalculation routines when internal belief collections change mid-execution.	`BDIAgent._revise_and_reconsider()` and `_reconsider()`: subscription-driven, three-valued check, DROP / SUSPEND / RESUME, then re-plan	`tests/bdi/test_reconsideration.py` (30 tests)
Deliverables: Complete source code showing the programmatic BDI objects	`ciphermesh/bdi/` (10 modules), public API in `ciphermesh/bdi/__init__.py`	Appendix B
Deliverables: ... and a test verification run demonstrating an intention shift when an unexpected environmental belief update hits the system.	`demo/bdi_verification_run.py` runs 9 scenarios + 2 ablations on the real engine	`logs/bdi_verification_run.log` (INTENTION SHIFT blocks), `bdi_summary.json`

3 DESIGN

3.1 Architecture

Figure 3.1 shows the BDI layer as built. Sensors and plan steps write beliefs; the deliberation cycle reads beliefs and desires, picks plan templates from the library and turns them into intentions; intention steps call the real engine, whose results flow back as new beliefs. Every arrow points one way, so the diagram is also the data-flow order of one cycle.

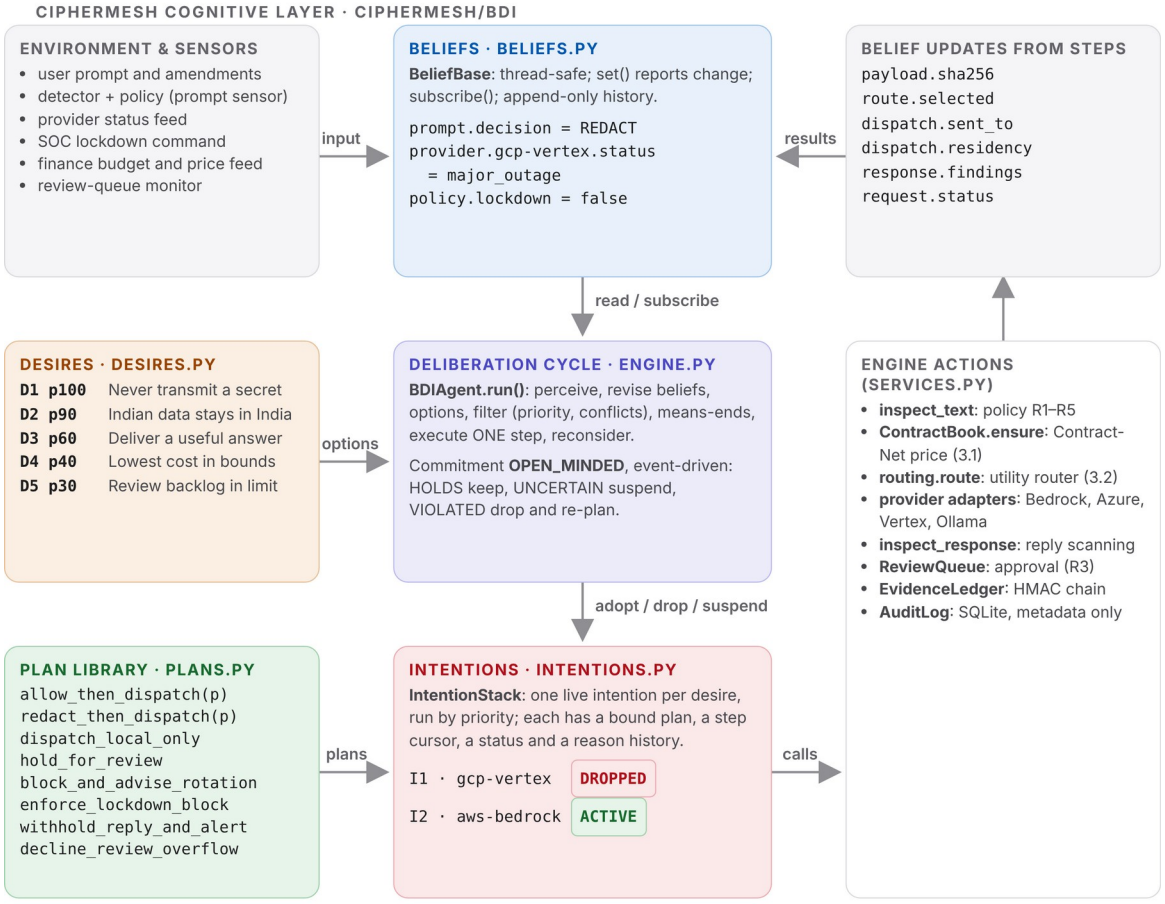


Figure 3.1: BDI architecture of the CipherMesh cognitive layer (package `ciphermesh/bdi`)

3.2 Background: the BDI model

Bratman describes intentions as *commitments* to partial plans [1]. Once an agent intends something it stops weighing every alternative again at every moment; the intention filters which new options are even considered. Cohen and Levesque formalised this as "intention is choice with commitment" [4]. Rao and Georgeff turned the idea into an abstract interpreter: options are generated from beliefs and desires, filtered into intentions, and executed step by step while new events are processed between steps [2]. Plan libraries with context conditions come from the Procedural Reasoning System (PRS) [5] and were later given a logical language, AgentSpeak(L) [6], which is implemented in the Jason platform [7]. Textbook treatments of the same loop appear in Wooldridge [8] and in Russell and Norvig [9].

The key design question is *how often to reconsider*. Rao and Georgeff name three commitment strategies: blind (keep the intention until it is believed achieved), single-minded (also drop it when it is believed impossible) and open-minded (also drop it when its goal is no longer desired) [10]. Kinny and Georgeff showed experimentally that the right amount of

reconsideration depends on how fast the world changes and on the cost of reconsidering [11]. CipherMesh uses the open-minded strategy but makes reconsideration *event-driven*, so its cost is paid only when a relevant belief actually changes.

3.3 Beliefs

A belief is one fact under a dotted key, with its source, a confidence in [0, 1], a timestamp and a per-key revision number. `BeliefBase.set()` returns True only when the value really changes, so repeated status reports do not wake the agent. Subscribers receive `(key, old, new, source)` after the lock is released, which lets a callback read the base without deadlock. The history is append-only and readers only get a copy. Raw prompt text is never stored as a belief; the text lives in a private per-request context and beliefs hold only hashes, categories, decisions, prices and statuses. Table 3.1 lists the belief families.

Table 3.1: Belief families used by the CipherMesh agent

Key (pattern)	Example value	Written by
`prompt.decision`, `prompt.categories`	REDACT, [aadhaar, email]	prompt sensor (`policy.inspect_text`)
`provider.<id>.status`	operational, partial_outage, major_outage	status feed, circuit breaker
`provider.<id>.residency`, `.region`	IN / ap-south-1	registry, provider health notices
`provider.<id>.price_per_1k`	2.28 (negotiated contract)	Contract-Net procurement, price feed
`budget.max_cost_per_1k`	6.00	route constraints, finance policy
`policy.lockdown`	true / false	agent state, SOC command
`review.queue_depth`, `review.limit`	25 / 25	review-queue monitor
`response.findings`, `response.decision`	[aws_access_key, CRITICAL]	response inspection step
`request.status`, `guard.secret_contained`	PENDING, COMPLETED, BLOCKED	plan steps

3.4 Desires

Desires are evaluated against beliefs on every cycle. An *achieve* desire receives an intention; a *maintain* desire never runs a plan of its own but constrains the plans chosen for lower-priority desires. Higher priority always wins, and D1 declares a conflict with D3 and D4, so whenever a secret risk is believed the delivery goal is pre-empted. Table 3.2 gives the five desires.

Table 3.2: CipherMesh desires (desires.py)

ID	Priority / kind	Description	Active when ... / satisfied when ...
D1	100 / achieve	Never transmit a secret (conflicts with D3, D4)	request pending and (secret in prompt, lockdown, or secret in reply) / secret contained
D2	90 / maintain	Keep Indian personal data in India	Aadhaar, PAN or Indian phone in prompt / no dispatch outside IN or LOCAL
D3	60 / achieve	Deliver a useful answer to the user	request pending / safe reply delivered
D4	40 / maintain	Lowest cost within quality bounds	request pending and a budget is known / effective price within budget
D5	30 / achieve	Keep the human-review backlog under its limit	REVIEW prompt pending / queue depth below limit or request deferred

3.5 Plans and the plan library

A plan template has an identifier, the desire it achieves, a context made of named conditions, an ordered list of steps and an optional *binder*. The binder of the dispatch plans asks the real router for every feasible provider and returns one binding per provider with its utility U, so means-ends reasoning is "best plan and best provider" in a single ranked list. Each step names the beliefs it relies on (`relevant_beliefs`) and may carry guard conditions. Conditions are three-valued: HOLDS, UNCERTAIN (wait, for example a partial outage) or VIOLATED. Table 3.3 lists the library.

Table 3.3: Plan library (plans.py)

Plan	Achieves	Context (all must HOLD)	Steps
allow_then_dispatch(p)	D3	ALLOW, no lockdown, p usable, residency allowed (D2), price within budget (D4)	prepare, procure, route, seal, dispatch, inspect, deliver
redact_then_dispatch(p)	D3	same, but prompt decision is REDACT	same seven steps; payload is the sanitized text
dispatch_local_only	D3	ALLOW or REDACT; local tier feasible	same seven steps on Ollama (local)
hold_for_review	D3	REVIEW, no lockdown, backlog has capacity	open_review_ticket, notify_user
block_and_advise_rotation	D1	critical secret in prompt	seal_evidence, advise_rotation, notify_user
enforce_lockdown_block	D1	lockdown is on	apply_lockdown, seal_evidence, notify_user
withhold_reply_and_alert	D1	critical secret in reply	discard_reply, seal_evidence, raise_alert, notify_user
decline_review_overflow	D5	REVIEW and backlog at limit	seal_evidence, notify_user

3.6 Intentions and their life cycle

An intention binds one plan and its parameters to one desire. It holds a cursor to the next step, a status and a reason history. The stack allows only one live intention per desire and always runs the live intention of the highest-priority desire first. Figure 3.2 shows the legal transitions; any other transition raises `IntentionError`.

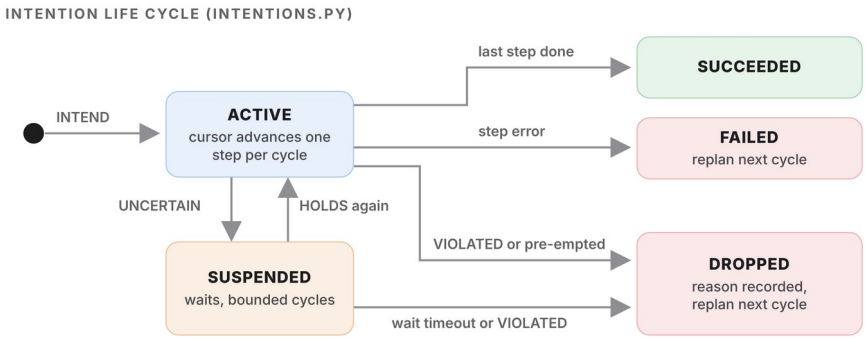


Figure 3.2: Intention state machine: ACTIVE, SUSPENDED and the three terminal states

3.7 The deliberation cycle

Figure 3.3 shows one cycle of `BDIAgent.run()`. The cycle executes exactly one plan step, so the agent gets a chance to reconsider between any two steps. The algorithm is:

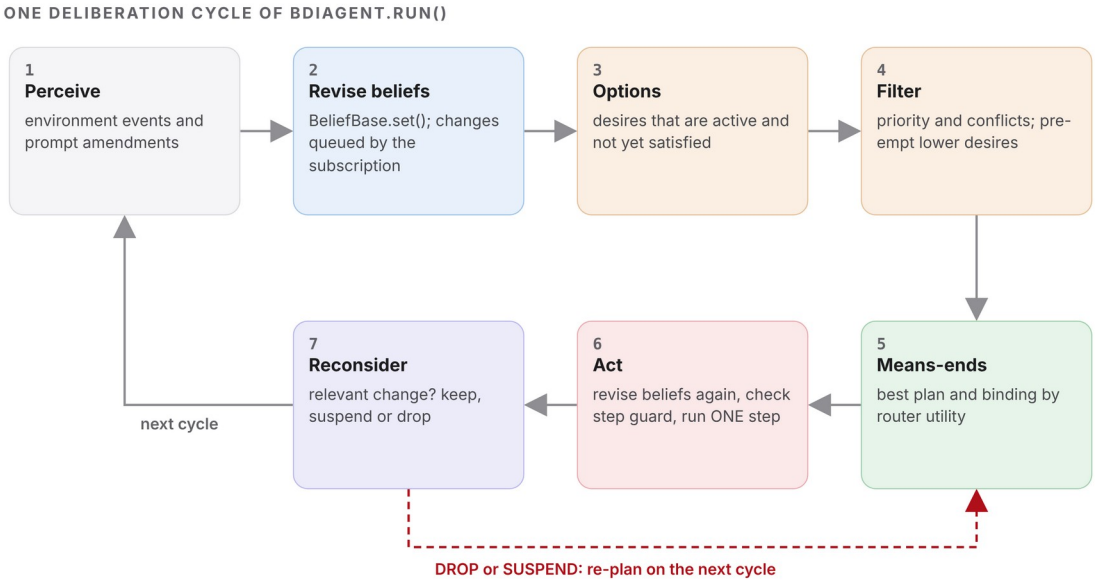


Figure 3.3: Deliberation cycle with the re-plan path (top) and the next-cycle path (bottom)

1. **Perceive.** Poll the environment for events due on this cycle (cycle or idle triggers) and apply them as belief updates.
2. **Revise beliefs.** Drain the queue filled by the belief subscription, log each change and run event-driven reconsideration (Section 3.8, steps 1–6).
3. **Generate options.** Every desire that is active and not yet satisfied is an option.
4. **Filter.** Walk the options in priority order; skip any option that conflicts with one already selected and record it as pre-empted. Drop the live intention of a pre-empted desire.
5. **Means-ends reasoning.** For each selected achieve desire without a live intention, ask the plan library for applicable (plan, binding) pairs, sorted by utility, then cost, then plan id. Adopt the best as a new intention; if a previous intention was dropped, log a REPLAN. If there is no applicable plan, mark the desire as having no plan for now.
6. **Act.** Just before the step, perceive again (events tied to that step) and revise beliefs once more, so nothing written during deliberation is left unprocessed. Then check the guard conditions of the step against the current belief base: if any guard does not HOLD, the step

is not executed and the intention is reconsidered instead. Otherwise run the step, which calls the real engine.

7. **Reconsider after the step** using the beliefs the step itself changed, then stop when no live intention and no achievable option remain.

3.8 Reconsideration and commitment strategy

Reconsideration is the routine that recalculates the plan. It is triggered by a belief change, never by a timer, and it only looks at intentions that depend on the changed key:

1. Build the *watch set* of the intention: the `relevant_beliefs` and guard reads of its **remaining** steps (completed steps no longer matter), plus the beliefs read by the desires. If no changed key matches, skip the intention and count the change as ignored.
2. Evaluate the guards of the remaining steps and the context conditions those steps still depend on. Each condition returns HOLDS, UNCERTAIN or VIOLATED.
3. Open-minded check: if the desire is no longer active, already satisfied, or pre-empted by a higher-priority conflicting desire, add a reason.
4. Any reason or VIOLATED condition: DROP the intention, record the reasons with the believed values, and open an *intention shift* that is closed by the next INTEND (the re-plan).
5. Otherwise any UNCERTAIN condition: SUSPEND the intention (or keep waiting, up to 4 cycles, then DROP with "suspension timeout").
6. Otherwise: keep the intention, or RESUME it if it was suspended.

Table 3.4 compares the three strategies supported by `CommitmentStrategy`. OPEN_MINDED is the production setting; BLIND is used only in the ablation of Section 5.

Table 3.4: Commitment strategies implemented in engine.py

Strategy	Drops an intention when	Use in this work
BLIND	a step fails (no reconsideration)	ablation baseline (S1-BLIND, S2-BLIND)
SINGLE_MINDED	a step fails, or its context or remaining guards are VIOLATED	tested in `test_single_minded_...`
OPEN_MINDED	as single-minded, or its desire is inactive, satisfied or pre-empted	default for all scenarios

3.9 Class and data model

Table 3.5 summarises the main classes, their key fields and the methods that the later Capstone integration will call.

Table 3.5: Class and data model of ciphermesh/bdi

Class	Key fields	Key methods
Belief / BeliefChange	key, value, source, confidence, updated_at, revision / seq, kind, old, new	as_dict()
BeliefBase	beliefs dict, history list, subscribers, RLock	set, get, has, retract, subscribe, snapshot, matching, history
Desire / DesireSet	id, description, priority, kind, conflicts_with, reads	is_active, is_satisfied, options, deliberate, constraining
Condition / PlanStep	description, check, reads / name, action, relevant_beliefs, guard	evaluate, describe / relevant_for
Plan / PlanLibrary	id, achieves, conditions, steps, binder	context, violations, bindings / options_for, rejected_for
Intention / IntentionStack	id, desire_id, plan, params, cursor, status, reasons, step_log	transition, advance, watched_beliefs / push, top, live
BDIAgent	beliefs, desires, library, intentions, commitment, tracer	perceive_world, submit, amend_prompt, run
EngineServices	agent, registry, contracts, ledger, transmissions	perceive_world, inspect, route_view, record_transmission

4 IMPLEMENTATION

4.1 Modules

The cognitive layer is a new subpackage. Table 4.1 lists its modules. The 112 engine tests from earlier units still pass unchanged.

Table 4.1: Modules of the BDI subpackage and demonstration

File	Purpose	Key classes / functions
`bdi/beliefs.py`	thread-safe belief repository with change subscription and history	Belief, BeliefChange, BeliefBase
`bdi/desires.py`	desires, option generation, priority and conflict filtering	Desire, DesireSet, ciphermesh_desires
`bdi/plans.py`	plan templates, three-valued conditions, step actions calling the engine	Condition, PlanStep, Plan, PlanLibrary
`bdi/intentions.py`	intention state machine and priority stack	Intention, IntentionStack, IntentionStatus
`bdi/engine.py`	deliberation cycle, reconsideration, intention shifts	BDIAgent, CommitmentStrategy, RunResult
`bdi/services.py`	bridge to detector, policy, procurement, router, providers, ledger	EngineServices, EvidenceLedger
`bdi/world.py`	shared interpretation of belief keys	allowed_residency, prompt_has_secret

File	Purpose	Key classes / functions
`bdi/trace.py`	JSONL + readable trace	Tracer
`bdi/scenarios.py`	scripted disturbances and scenario catalogue	Disturbance, Trigger, Environment
`demo/bdi_verification_run.py`	runs all scenarios, writes the three logs	main, shift_block, summarise

4.2 Selected code

Listing 1 is the core of belief revision. The change test and the history append happen under the lock; subscribers are notified after the lock is released.

```
def set(self, key, value, source, confidence=1.0) -> bool:
    self._validate(key, source, confidence)
    value = copy.deepcopy(value)
    with self._lock:
        previous = self._beliefs.get(key)
        changed = previous is None or previous.value != value
        revision = 1 if previous is None else previous.revision + (1 if changed else 0)
        self._beliefs[key] = Belief(key=key, value=value, source=source,
                                    confidence=float(confidence), updated_at=_now(),
                                    revision=revision)
        if not changed:
            return False
        ...  # append BeliefChange to the history, copy the subscriber list
    self._notify(callbacks, key, old_value, value, source)
    return True
```

Listing 1: BeliefBase.set() in beliefs.py (abridged)

Listing 2 shows how one plan template is declared. The context conditions are shared objects, so the same condition text appears in the plan, in the step guards and in the drop reasons of the log.

```
Plan("allow_then_dispatch", "D3",
     "send the clean prompt to the best feasible cloud provider",
     (C_PENDING, C_NO_LOCKDOWN, C_ALLOW, C_USABLE, C_RESIDENCY, C_BUDGET,
      C_REPLY_CLEAN),
     _dispatch_steps(), binder=_cloud_binder("allow"))

PlanStep("dispatch", step_dispatch, R_DISPATCH, guard=G_DISPATCH, transmits=True)
C_USABLE = Condition("provider {provider} is usable", _verdict_status,
                     frozenset({"provider.{provider}.status"}))
```

Listing 2: a plan template, its transmitting step and a condition (plans.py)

Listing 3 is the event-driven part of reconsideration. Only intentions whose watch set matches a changed key are evaluated.

```
for intention in live:
    watched = intention.watched_beliefs()
    if self.commitment == CommitmentStrategy.OPEN_MINDED:
        watched |= self._desire_reads
    relevant = [c for c in changes if _matches(c["key"], watched)]
    waiting = (intention.status == IntentionStatus.SUSPENDED
               and intention.suspended_cycle != self.cycle)
    due = bool(relevant) or (origin == "step" and (intention is executed or waiting))
    if not due:
        continue
    self.stats["evaluated"] += 1
    self._reconsider(intention, relevant)
```

Listing 3: selecting intentions to reconsider (engine.py)

Deliberation decides what the agent *should* do; it is not the only line of defence. The `dispatch` step re-checks the safety rules itself before any byte leaves the host (Listing 4): lockdown, residency, a provider believed to be down, and a payload built from an older prompt. The ablation in Section 5 shows why this matters: a blind agent that never reconsiders is still stopped by this interlock.

```
def step_dispatch(ctx: StepContext) -> StepResult:
    b, svc, pid = ctx.beliefs, ctx.services, ctx.provider
    residency = b.get(world.pkey(pid, "residency"))
    if svc.agent.state.force_block_all or world.lockdown(b):
        return _fail("interlock: emergency lockdown is active; nothing transmitted")
    if residency not in world.allowed_residency(b):
        return _fail(f"interlock: residency {residency} not allowed ...")
    if world.provider_status(b, pid) in world.STATUS_DOWN:
        return _fail(f"interlock: {pid} is believed ...; nothing transmitted")
    if (ctx.request.payload is None
            or b.get("payload.prompt_sha256") != b.get("prompt.sha256")):
        return _fail("interlock: payload was prepared from an older prompt; ...")
```

Listing 4: safety interlocks inside the transmitting step (plans.py, abridged)

4.3 Invariants enforced in code

- **No step runs on stale beliefs.** Pending belief changes are drained and reconsidered immediately before every step, and the guards of the step are evaluated at execution time; a failing guard triggers reconsideration instead of the step (tested by writing an outage belief during deliberation and after the last revision).
- **Nothing leaves during lockdown or to a non-Indian region for Indian data.** Checked by deliberation and again by the dispatch interlock.
- **No raw prompt text or secret value in beliefs, logs or the trace.** Only hashes, categories and decisions; a test greps every trace for the test secrets.
- **Every adopted and dropped intention is audited** in the engine SQLite audit log and sealed evidence is kept in an HMAC-SHA256 hash chain [12].
- **Determinism.** Option ranking ties break on cost and then plan id; scenario runs produce identical event sequences.

5 VERIFICATION RUN

5.1 Set-up

The verification run executes nine scenarios and two ablation runs with `python demo/bdi_verification_run.py`. Each run builds a fresh engine: real detector and policy, Contract-Net negotiation for four providers, the constrained router, simulated provider adapters

(latency scale 0.05 of the profile p95, so a cloud call takes about 30 ms and the local tier about 60 ms), response inspection, the review queue and the HMAC evidence ledger. Disturbances are injected through `Environment` at a chosen step, which is the "unexpected environmental belief update" required by the brief. Result: **11/11 runs met their expected behaviour**. Table 5.1 lists the scenarios and where each disturbance strikes.

Table 5.1: Scenario catalogue and injection points (scenarios.py)

ID	Disturbance	Injected	Expected shift
S0	none (control run)	-	no shift
S1	GCP Vertex major outage	before 'seal_dispatch_record'	gcp-vertex to aws-bedrock
S2	emergency lockdown from the SOC	before 'dispatch'	dispatch (D3) to block (D1)
S3	provider reply echoes an AWS access key	before 'dispatch'	deliver (D3) to withhold (D1)
S4	price spike to $6.80, then budget cut to $1.00	before 'seal', then 'dispatch'	gcp to aws, then aws to local
S5	Bedrock region moves to us-east-1 (Aadhaar prompt)	before 'dispatch'	aws-bedrock to azure-ai (IN)
S6	user amends prompt and adds a PAN	before 'seal_dispatch_record'	allow (US) to redact (IN)
S7	partial outage, recovered two idle cycles later	before 'dispatch'	suspend, then resume (no shift)
S8	25 review tickets arrive in a burst	before 'open_review_ticket'	hold (D3) to decline (D5)

5.2 Baseline: no disturbance (S0)

With no belief change the agent adopts `allow_then_dispatch(gcp-vertex)` (U = 77.93, the router winner) and runs its seven steps without a single drop: 7 steps in 7 cycles, 34.1 ms, one transmission. Every later scenario starts from this same intention, so any change of plan in those runs is caused by the injected belief update and nothing else.

5.3 S1: provider outage mid-dispatch

Listing 5 is the trace of S1 (belief lines not related to the trigger are omitted). After three steps the status feed reports a major outage. The change matches the watch set of I1 (its `seal` and `dispatch` steps depend on `provider.gcp-vertex.status`), the condition "provider gcp-vertex is usable" is VIOLATED, and I1 is dropped with 3 of 7 steps done. On the next cycle means-ends reasoning runs the router again on the revised beliefs and adopts AWS Bedrock.

```
+  4.35 c00 PERCEIVE   request req-s1 received: 82 bytes, policy ALLOW
+  5.19 c01 INTEND     I1 adopted for D3: allow_then_dispatch(gcp-vertex) U=77.93 [pr...
+  6.78 c04 PERCEIVE   environment: status feed reports GCP Vertex AI us-central1 maj...
+  6.81 c04 BELIEF     provider.gcp-vertex.status: operational -> major_outage (sourc...
+  6.96 c04 RECONSIDER I1 invalidated by ['provider.gcp-vertex.status']
+  6.98 c04 DROP       I1 allow_then_dispatch(gcp-vertex) dropped before 'seal_dispat...
+  7.75 c05 INTEND     I2 adopted for D3: allow_then_dispatch(aws-bedrock) U=77.28 [p...
+  7.82 c05 REPLAN     I1 allow_then_dispatch(gcp-vertex) [DROPPED] -> I2 allow_then_...
+ 42.82 c09 RECONSIDER I2 kept after ['response.received']: 2 remaining condition(s) ...
+ 43.34 c10 RECONSIDER I2 kept after ['response.decision']: 2 remaining condition(s) ...
+ 43.89 c11 FINISH     request req-s1 finished: COMPLETED after 11 cycles, 10 steps, ...
```

Listing 5: S1 trace excerpt from logs/bdi_trace.jsonl

The log also prints a dedicated block for every change of mind. Listing 6 is the block for S1, copied from `logs/bdi_verification_run.log`. The reaction time from the drop to the new intention is below one millisecond because the cached contracts and the router are cheap; the cost of the outage is the three repeated preparation steps, not waiting.

```
====================== INTENTION SHIFT 1/1 (cycle 4, +7.04 ms) ======================
BEFORE   I1 allow_then_dispatch(provider=gcp-vertex)  desire D3  U=77.93  progress
         3/7  next step 'seal_dispatch_record'
TRIGGER  provider.gcp-vertex.status: operational -> major_outage  [source
         status-feed:gcp-vertex, confidence 0.98]
DROPPED  I1: requires 'provider gcp-vertex is usable' but believed
         provider.gcp-vertex.status=major_outage
AFTER    I2 allow_then_dispatch(provider=aws-bedrock)  desire D3  U=77.28  adopted at
         cycle 5 (+7.81 ms, reaction 0.77 ms)
=====================================================================================
```

Listing 6: INTENTION SHIFT block for S1

Figure 5.1 is drawn from the same trace. Each box is one executed step; the dashed line is the arrival of the belief.

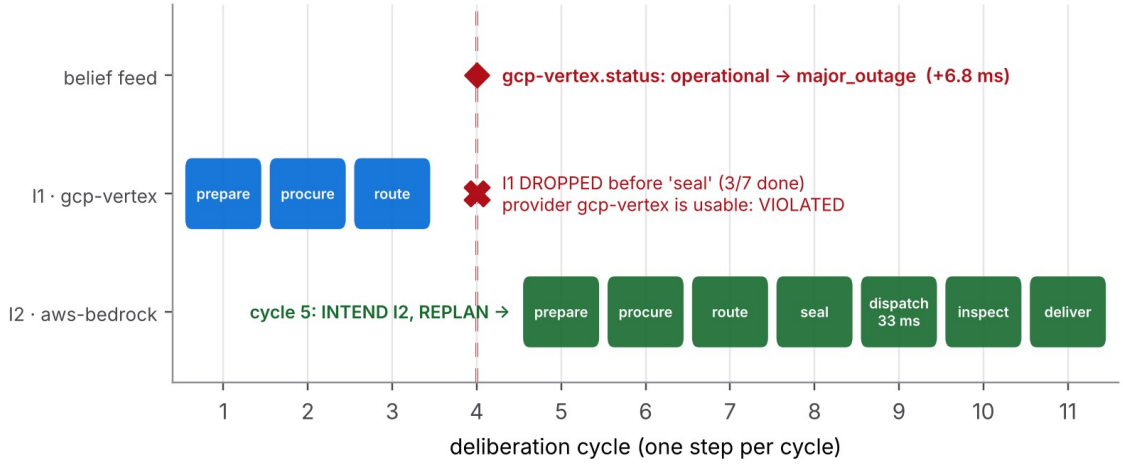


Figure 5.1: S1 swimlane: intention I1 is dropped at the outage belief and I2 completes on AWS Bedrock

5.4 S2: emergency lockdown mid-execution

The lockdown arrives just before the transmitting step. Two independent reasons drop I1: D1 becomes an option and pre-empts D3 (open-minded reason), and the context condition "policy lockdown is off" is VIOLATED (single-minded reason). The block plan then pushes `force_block_all` into the engine and re-inspects the prompt with the real policy, which now answers BLOCK. Transmissions in this run: **0**.

```
====================== INTENTION SHIFT 1/1 (cycle 5, +9.22 ms) ======================
BEFORE   I1 allow_then_dispatch(provider=gcp-vertex)  desire D3  U=77.93  progress
         4/7  next step 'dispatch'
TRIGGER  policy.lockdown: false -> true  [source soc:command_instruction, confidence
         1.00]
DROPPED  I1: pre-empted by D1 'Never transmit a secret' (priority 100)
         I1: requires 'policy lockdown is off' but believed policy.lockdown=True
AFTER    I2 enforce_lockdown_block  desire D1  U=100.00  adopted at cycle 6 (+9.51
         ms, reaction 0.29 ms)
=====================================================================================
```

Listing 7: INTENTION SHIFT block for S2

5.5 S3: the provider reply leaks a secret

Here the trigger is produced by the agent itself. The provider starts echoing a cached staging credential; the `inspect_response` step runs the real detector on the reply and writes `response.findings` with a CRITICAL `aws_access_key`. D1 activates on that belief and the deliver step is never run. The new intention discards the reply from memory, seals evidence, raises an alert (`threat.level = high`, audit event `bdi_security_alert`) and tells the user the reply was withheld. The secret value never appears in any log; only its category does.

```
====================== INTENTION SHIFT 1/1 (cycle 6, +37.34 ms) ======================
BEFORE   I1 allow_then_dispatch(provider=gcp-vertex)  desire D3  U=77.93  progress
         6/7  next step 'deliver'
TRIGGER  response.findings: [] -> [{"category": "aws_access_key", "severity":
         "CRITICAL", "rule": "D1"}]  [source intention:I1/inspect_response,
         confidence 1.00]
TRIGGER  response.decision: null -> BLOCK  [source intention:I1/inspect_response,
         confidence 1.00]
DROPPED  I1: pre-empted by D1 'Never transmit a secret' (priority 100)
         I1: requires 'reply contains no critical secret (D1)' but believed
         response.findings=[{'category': 'aws_access_key', 'severity': 'CRITICAL',
         'rule': 'D1'}]
AFTER    I2 withhold_reply_and_alert  desire D1  U=100.00  adopted at cycle 7 (+37.70
         ms, reaction 0.35 ms)
=======================================================================================
```

Listing 8: INTENTION SHIFT block for S3

5.6 S4: price spike, then budget cut

S4 shows that reconsideration can fire more than once in one request. Listing 9 shows both shifts. The first belief (GCP price $6.80 above the $6.00 budget) violates the D4 condition of I1, so the router picks AWS Bedrock. Finance then cuts the budget to $1.00, which violates the same condition for I2; no cloud provider is now feasible and the agent falls back to the local tier at $0.33 per 1k requests.

```
+  5.58 c01 INTEND     I1 adopted for D3: allow_then_dispatch(gcp-vertex) U=77.93 [pr...
+  7.16 c04 BELIEF     provider.gcp-vertex.price_per_1k: 1.79 -> 6.8 (source procurem...
+  7.35 c04 DROP       I1 allow_then_dispatch(gcp-vertex) dropped before 'seal_dispat...
+  8.18 c05 INTEND     I2 adopted for D3: allow_then_dispatch(aws-bedrock) U=77.28 [p...
+  9.91 c09 BELIEF     budget.max_cost_per_1k: 6.0 -> 1.0 (source finance:budget-policy)
+ 10.07 c09 DROP       I2 allow_then_dispatch(aws-bedrock) dropped before 'dispatch' ...
+ 10.74 c10 INTEND     I3 adopted for D3: dispatch_local_only(local) U=64.65 [prepare...
```

Listing 9: S4 trace excerpt (two drops, three intentions)

5.7 S5 and S6: Indian data residency

In S5 the prompt contains an Aadhaar number and an e-mail address, so the policy answers REDACT and D2 limits dispatch to IN or LOCAL. The agent intends `redact_then_dispatch(aws-bedrock)`. Before the transmission, a provider health notice says Bedrock now serves from us-east-1. The residency condition is VIOLATED and the agent re-plans to Azure AI Foundry in centralindia (Listing 10).

```
====================== INTENTION SHIFT 1/1 (cycle 5, +8.09 ms) ======================
BEFORE   I1 redact_then_dispatch(provider=aws-bedrock)  desire D3  U=77.28  progress
         4/7  next step 'dispatch'
TRIGGER  provider.aws-bedrock.residency: IN -> US  [source
         aws-health:inference-profile, confidence 0.97]
DROPPED  I1: requires 'provider aws-bedrock residency allowed for this prompt (D2)'
         but believed prompt.categories=['aadhaar', 'email'],
         provider.aws-bedrock.residency=US
AFTER    I2 redact_then_dispatch(provider=azure-ai)  desire D3  U=76.84  adopted at
         cycle 6 (+8.88 ms, reaction 0.79 ms)
=============================================================================================
```

Listing 10: INTENTION SHIFT block for S5

S6 starts with a clean prompt routed to GCP Vertex (US). The user then edits the prompt and adds a PAN. The prompt sensor re-inspects the new text, D2 becomes active in the middle of execution, and two conditions of I1 fail at once: the prompt is no longer ALLOW and US residency is no longer allowed. The new intention redacts the PAN and sends the sanitized text to AWS Bedrock in India. In both runs every transmission was to IN residency and was redacted, which `test_residency_is_never_violated` asserts.

```
+  5.14 c01 INTEND     I1 adopted for D3: allow_then_dispatch(gcp-vertex) U=77.93 [pr...
+  6.91 c04 BELIEF     prompt.categories: [] -> ['pan'] (source user:amendment)
+  6.92 c04 BELIEF     prompt.decision: ALLOW -> REDACT (source user:amendment)
+  6.95 c04 DESIRE     D2 'Keep Indian personal data in India' (p90, maintain): inact...
+  7.17 c04 DROP       I1 allow_then_dispatch(gcp-vertex) dropped before 'seal_dispat...
+  8.08 c05 INTEND     I2 adopted for D3: redact_then_dispatch(aws-bedrock) U=77.28 [...
+ 42.85 c09 STEP       I2 redact_then_dispatch(aws-bedrock) [5/7] dispatch -> OK: tra...
```

Listing 11: S6 trace excerpt (mid-execution activation of D2)

5.8 S7 and S8: suspension and a shift across desires

Not every belief change should cause a new plan. In S7 the status feed reports a *partial* outage, which the provider-status condition maps to UNCERTAIN. The intention is suspended instead of dropped, waits two idle cycles, and resumes at the same `dispatch` step when the feed reports recovery. No step is repeated. S8 shows a shift between two different desires: a REVIEW prompt is about to open a review ticket when a burst fills the queue to its limit of 25. `hold_for_review` is dropped, D3 has no applicable plan, and D5 adopts `decline_review_overflow`, which fails closed without sending anything.

```
+  7.98 c05 BELIEF     provider.gcp-vertex.status: operational -> partial_outage (sou...
+  8.11 c05 SUSPEND    I1 allow_then_dispatch(gcp-vertex) suspended at dispatch: wait...
+ 28.80 c06 RECONSIDER I1 still suspended (provider gcp-vertex is usable UNCERTAIN; w...
+ 28.95 c07 BELIEF     provider.gcp-vertex.status: partial_outage -> operational (sou...
+ 29.15 c07 RESUME     I1 allow_then_dispatch(gcp-vertex) resumed at dispatch (all co...
+ 58.79 c07 STEP       I1 allow_then_dispatch(gcp-vertex) [5/7] dispatch -> OK: trans...
```

Listing 12: S7 trace excerpt (suspend, wait, resume)

5.9 Summary of all scenarios

Table 5.2 is generated from `logs/bdi_summary.json`. Figure 5.2 places every run on a common cycle axis, so the position of each disturbance, drop and new intention can be compared at a glance.

Table 5.2: Summary of the verification run (open-minded commitment)

ID	Initial intention	Trigger belief	Adopted intention(s)	Outcome	Steps
S0	allow(gcp-vertex)	none	same (no shift)	COMPLETED	7
S1	allow(gcp-vertex)	gcp-vertex.status = major_outage	allow(aws-bedrock)	COMPLETED	10
S2	allow(gcp-vertex)	policy.lockdown = true	enforce_lockdown_block	BLOCKED	7
S3	allow(gcp-vertex)	response.findings = aws_access_key (CRITICAL)	withhold_reply_and_alert	WITHHELD	10
S4	allow(gcp-vertex)	gcp-vertex.price_per_1k = 6.8	allow(aws-bedrock), then local_only(local)	COMPLETED	14
S5	redact(aws-bedrock)	aws-bedrock.residency = US	redact(azure-ai)	COMPLETED	11
S6	allow(gcp-vertex)	prompt.categories = [pan]	redact(aws-bedrock)	COMPLETED	10
S7	allow(gcp-vertex)	gcp-vertex.status = partial_outage	same (suspend, resume)	COMPLETED	7
S8	hold_for_review	review.queue_depth = 25	decline_review_overflow	DEFERRED	2

allow(p), redact(p) and local_only(p) abbreviate allow_then_dispatch, redact_then_dispatch and dispatch_local_only; WITHHELD is the status RESPONSE_WITHHELD.

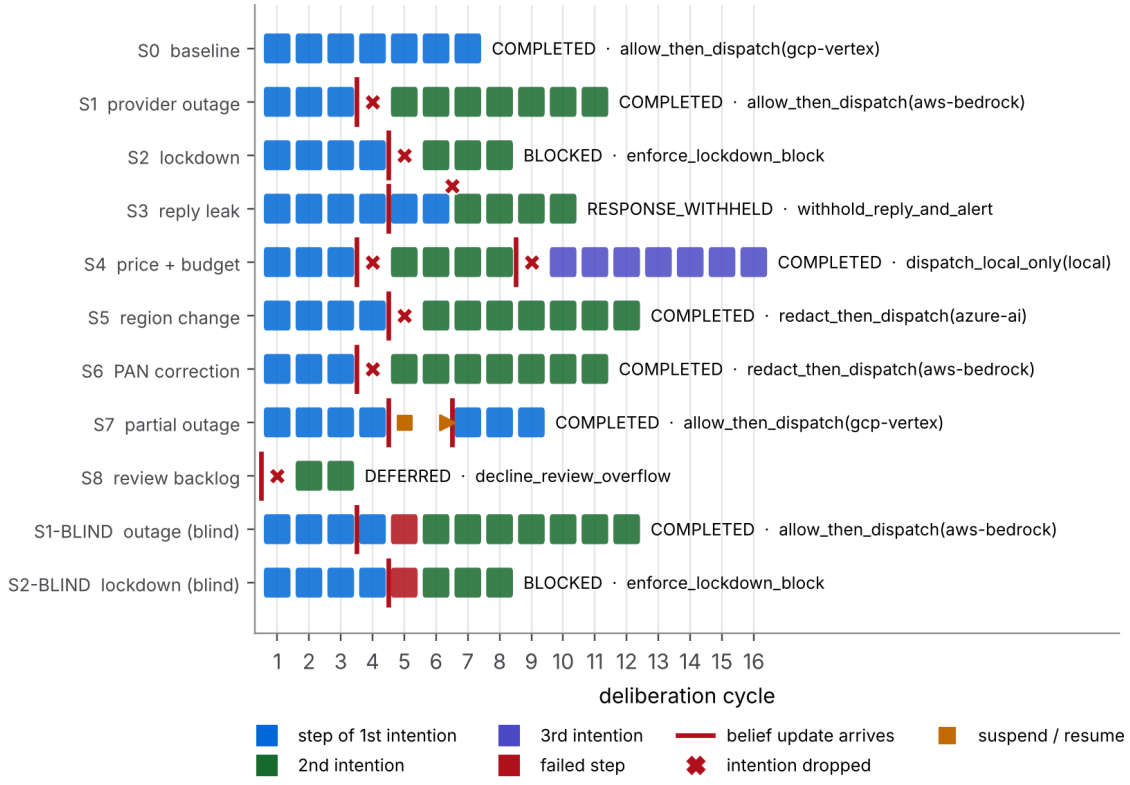


Figure 5.2: All runs on one cycle axis: steps per intention, disturbances, drops and suspensions

5.10 Ablation: why reconsideration matters

To show that the shift is caused by reconsideration and not by luck, S1 and S2 were repeated with BLIND commitment. Table 5.3 and Figure 5.3 compare the runs. In S1-BLIND the agent ignores the outage belief and keeps executing the GCP intention until its `dispatch` step, which the interlock refuses because the provider is believed down. The intention FAILS and only then is the request re-planned: 12 steps instead of 10, with one failed step. In S2-BLIND the stale dispatch intention is still alive when the lockdown arrives and is likewise stopped only by the interlock. The open-minded agent never reaches either point, because it drops the intention as soon as the belief arrives.

Table 5.3: Ablation: open-minded versus blind commitment

Run	Commitment	Provider calls	Failed steps	Steps	How the first intention ended
S1	OPEN_MINDED	1	0	10	DROPPED before seal_dispatch_record
S1-BLIND	BLIND	1	1	12	FAILED at dispatch
S2	OPEN_MINDED	0	0	7	DROPPED before dispatch
S2-BLIND	BLIND	0	1	8	FAILED at dispatch

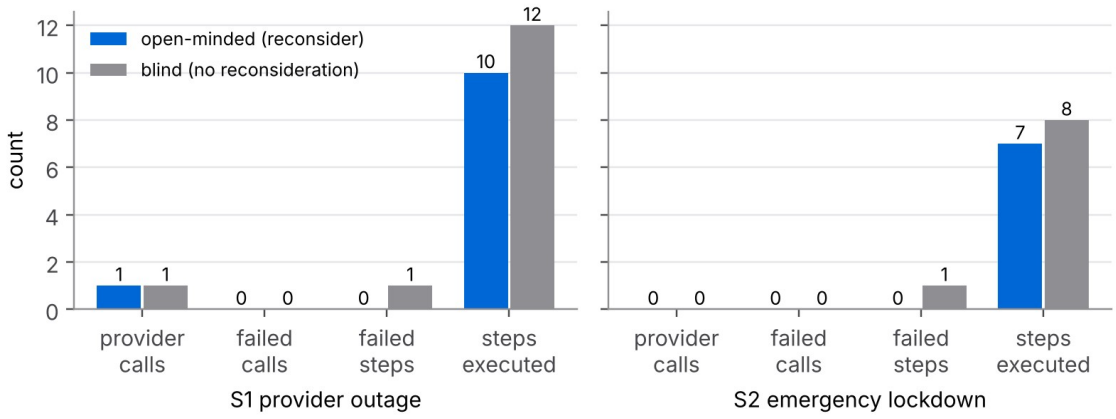


Figure 5.3: Ablation counts from logs/bdi_summary.json

6 TESTING

The full suite (engine and BDI) was run with `python -m pytest tests -vv -p no:cacheprovider > logs/pytest_results.txt` from a copy of the submission folder; the saved file is the unedited output. **188 tests passed**: the 112 engine tests from earlier units and 76 new BDI tests. Table 6.1 lists the new test files.

Table 6.1: BDI test files and what they cover

File	Tests	What is verified
`test_beliefs.py`	13	change detection, revisions, append-only history, subscriptions, retraction, validation, 12-thread stress test
`test_desires.py`	12	priority order, D1 / D2 / D5 activation triggers, conflicts and tie-break
`test_plans.py`	10	router-ranked options, IN/LOCAL only for Indian data, budget and outage filters, three-valued context
`test_intentions.py`	4	legal / illegal transitions, cursor, shrinking watch set, stack priority
`test_reconsideration.py`	30	every scenario, no transmission after lockdown, residency never violated, suspension timeout, guard check at execution time, blind and single-minded strategies
`test_trace_and_determinism.py`	7	determinism, all event types present, no secret in any trace, valid JSONL, ledger tamper detection, audit entries

Edge cases covered

- A belief change from a monitor thread during step execution is processed before the next step (`test_belief_changes_from_other_threads_...`).
- A provider outage *after* the payload is sent does not drop the intention, because no remaining step depends on that belief.
- An unrelated price change for another provider is counted as ignored and causes no reconsideration.
- A partial outage that never recovers turns into a drop after 4 cycles and a re-plan to the next provider.
- A faulty subscriber cannot break belief revision; its error is recorded and other subscribers still run.

7 SUBMISSION ANALYSIS

Table 7.1 checks each requirement against its evidence.

Table 7.1: Requirement, evidence and status

Requirement	Evidence	Status
Three explicit BDI collections / classes	`BeliefBase`, `DesireSet`, `PlanLibrary` + `IntentionStack` in `ciphermesh/bdi/`	**PASS**
Cognitive-layer state machine	intention states (Figure 3.2) and the deliberation cycle (Figure 3.3)	**PASS**
Dynamic plan recalculation on belief change	reconsideration in `engine.py`; 10 intention shifts across S1 to S8 and the two ablations	**PASS**
Verification run with an intention shift after an unexpected belief update	`logs/bdi_verification_run.log` INTENTION SHIFT blocks; summary JSON	**PASS**
Baseline proves causality	S0 completes unchanged; S1/S2 blind ablation	**PASS**
Tests for every new module	76 BDI tests, 188 in total, all passing	**PASS**

7.1 Strengths

- The plans call the real engine, so the run exercises detector, policy, procurement, router, adapters, review queue and audit together.
- Reconsideration is event-driven and scoped to the remaining steps, which keeps its cost low and avoids pointless drops after a payload is already sent.
- Three-valued conditions separate "wait" from "give up", which a two-valued BDI context cannot express.
- Every change of mind is explained with the triggering belief, its source and confidence, and the believed values that made a condition fail.
- Safety does not depend on deliberation alone: the dispatch interlock blocks transmission even for a blind agent.

7.2 Limitations

- Cloud providers are simulated adapters; prices and latencies are illustrative planning numbers, not measurements. The local tier can call a real Ollama server only when it is enabled.
- Disturbances are scripted at chosen steps. A production system would feed beliefs from live status pages and price APIs.
- The agent handles one request at a time. Several requests would need either one BDIAgent per request or a shared belief base with per-request intention stacks.
- Plans are fixed templates with one level of steps; there are no sub-goals that post new desires, as AgentSpeak plans can.
- Re-planning restarts the new pipeline from its first step. Reusing finished work (for example the already prepared payload) would save a few steps.

7.3 How this feeds the Capstone (Assignment 5.2)

The public API (`BeliefBase`, `DesireSet`, `PlanLibrary`, `IntentionStack`, `BDIAgent`, `EngineServices`) is exported from `ciphermesh/bdi/__init__.py`. In the Capstone, the gateway will submit each request to a `BDIAgent`; the runtime monitor from Unit 2.1 will write provider health into the belief base; the Unit 2.2 COMMAND_INSTRUCTION channel will set `policy.lockdown`; and the existing audit log already stores every adopted and dropped intention for the dashboard.

8 CONCLUSION

This assignment gave CipherMesh an explicit cognitive layer. Beliefs, desires and intentions are separate, tested classes; a deliberation cycle turns them into one action per cycle; and reconsideration recalculates the plan as soon as a relevant belief changes. The verification run shows the required intention shift in every disturbed scenario: an outage moves the request to another provider, a lockdown or a leaking reply hands control to the "never transmit a secret" goal, a price spike and a budget cut move the request down to cheaper tiers, and a change in data residency keeps Indian data in India. A control run without disturbance and a blind-commitment ablation show that these shifts are caused by the belief updates and by reconsideration. All tests pass and every number in this report comes from the saved logs.

REFERENCES

[1] M. E. Bratman, *Intention, Plans, and Practical Reason*. Cambridge, MA, USA: Harvard University Press, 1987.

[2] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in *Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS-95)*, San Francisco, CA, USA, 1995, pp. 312–319.

[3] R. G. Smith, "The Contract Net Protocol: High-level communication and control in a distributed problem solver," *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, Dec. 1980.

[4] P. R. Cohen and H. J. Levesque, "Intention is choice with commitment," *Artificial Intelligence*, vol. 42, no. 2–3, pp. 213–261, 1990.

[5] M. P. Georgeff and A. L. Lansky, "Reactive reasoning and planning," in *Proc. 6th National Conf. Artificial Intelligence (AAAI-87)*, Seattle, WA, USA, 1987, pp. 677–682.

[6] A. S. Rao, "AgentSpeak(L): BDI agents speak out in a logical computable language," in *Agents Breaking Away: Proc. 7th European Workshop on Modelling Autonomous Agents in a Multi-Agent World (MAAMAW'96)*, W. Van de Velde and J. W. Perram, Eds., LNCS vol. 1038. Berlin, Germany: Springer, 1996, pp. 42–55.

[7] R. H. Bordini, J. F. Hübner, and M. Wooldridge, *Programming Multi-Agent Systems in AgentSpeak using Jason*. Chichester, U.K.: John Wiley & Sons, 2007.

[8] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: John Wiley & Sons, 2009.

[9] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[10] A. S. Rao and M. P. Georgeff, "Modeling rational agents within a BDI-architecture," in *Proc. 2nd Int. Conf. Principles of Knowledge Representation and Reasoning (KR-91)*, Cambridge, MA, USA, 1991, pp. 473–484.

[11] D. N. Kinny and M. P. Georgeff, "Commitment and effectiveness of situated agents," in *Proc. 12th Int. Joint Conf. Artificial Intelligence (IJCAI-91)*, Sydney, Australia, 1991, pp. 82–88.

[12] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, Feb. 1997.

APPENDIX A HOW TO RUN

All commands are run from the submission folder with the CipherMesh Python environment (Python 3.13, see `requirements.txt`).

```
# 1. verification run: writes logs/bdi_verification_run.log,
#    logs/bdi_trace.jsonl and logs/bdi_summary.json
python demo/bdi_verification_run.py --latency-scale 0.05 --idle-wait 0.02

# 2. full test suite (writes the file used in Section 6)
python -m pytest tests -vv -p no:cacheprovider > logs/pytest_results.txt

# 3. only the BDI tests
python -m pytest tests/bdi -q

# 4. report figures (needs matplotlib; diagrams are rendered from diagrams.html)
python docs/figures/make_charts.py
```

Listing 13: commands that produce every artefact

A minimal use of the API from other code is shown in Listing 14.

```
from ciphermesh.bdi import BDIAgent, EngineServices
agent = BDIAgent(EngineServices())
agent.perceive_world()
agent.submit("Summarise Q3 cloud spend drivers.", request_id="req-1")
result = agent.run()      # RunResult: final_status, shifts, transmissions, ...
print(result.final_status, [s.adopted["label"] for s in result.shifts])
```

Listing 14: using the BDI agent from code

APPENDIX B FILE LISTING

Table B.1: Files in the submission

Path	Content
`README.md`, `requirements.txt`, `pytest.ini`	overview, deliverables map, commands; dependencies; test settings
`ciphermesh/` (engine)	Units 1 to 3 engine: detector, policy, agent, review, procurement, routing, providers, runtime, transport
`ciphermesh/bdi/`	beliefs, desires, plans, intentions, engine, services, world, trace, scenarios, __init__
`demo/bdi_verification_run.py`	verification run
`tests/engine/` (112 tests)	regression tests of earlier units
`tests/bdi/` (76 tests)	BDI unit and scenario tests, conftest
`logs/bdi_verification_run.log`	readable timestamped run with INTENTION SHIFT blocks
`logs/bdi_trace.jsonl`, `logs/bdi_summary.json`	machine-readable trace and per-scenario summary
`logs/pytest_results.txt`	full test output
`docs/Assignment_4.1_*_Report.docx`, `.pdf`	this report (BDI Goal-Driven Modeling)
`docs/figures/`	figure PNGs, `diagrams.html` source, `make_charts.py`
`docs/report_src/content.js`	report source